

# **Sim Documentation and Reference**

Reuben Georgi

June 27, 2026

# Contents

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                 | <b>3</b>  |
| <b>2</b> | <b>Setup &amp; Run with main.py</b> | <b>4</b>  |
| <b>3</b> | <b>Parameter Files</b>              | <b>5</b>  |
| <b>4</b> | <b>Requirements</b>                 | <b>6</b>  |
| <b>5</b> | <b>Design</b>                       | <b>7</b>  |
| <b>6</b> | <b>Implementation</b>               | <b>8</b>  |
| <b>7</b> | <b>Usage</b>                        | <b>9</b>  |
| <b>8</b> | <b>Conclusion</b>                   | <b>10</b> |

# 1 Introduction

This Python project allows users to simulate drone flight with provisions for choosing different control techniques and tweak simulation parameters in order to examine their effects. The intention is to provide a relatively lightweight tool which is designed to be modular and extensible. While it is still a work in progress in terms of simulation fidelity and structure, many features are already available to get started.

The project consists of the following major components:

- Planners
- Controllers
- Simulators
- Visualizers

## 2 Setup & Run with main.py

(Identify dependencies with their version and list them here)

The "main.py" file provides an easy way to get started with the simulation tool and also displays how the project is setup. From the imports, one can see the available the planner and controller options. All parameters in the project are set with the help of yaml files and parameters related to the conditions to be simulated need to be loaded first. Some examples of the parameters include simulation time step, quadcopter parameters (such as mass, arm length etc.), time stamped waypoints and positions of obstacles. Use "default.yaml" as a reference and to experiment.

All classes defined in the project expect the simulation parameters (loaded as a dictionary) to be provided as the first argument when instantiating objects of the class. All planners implement a calculate\_trajectory method which returns the trajectory itself and a "trajectory object" which must be passed to the controller with the set\_trajectory method. The rest of the script is self explanatory

## 3 Parameter Files

## 4 Requirements

List functional and non-functional requirements.

# 5 Design

System architecture, diagrams, decisions.

## 6 Implementation

Explain major parts of your implementation.

# 7 Usage

Describe how to run or use the project.

## 8 Conclusion

Final notes and future improvements.